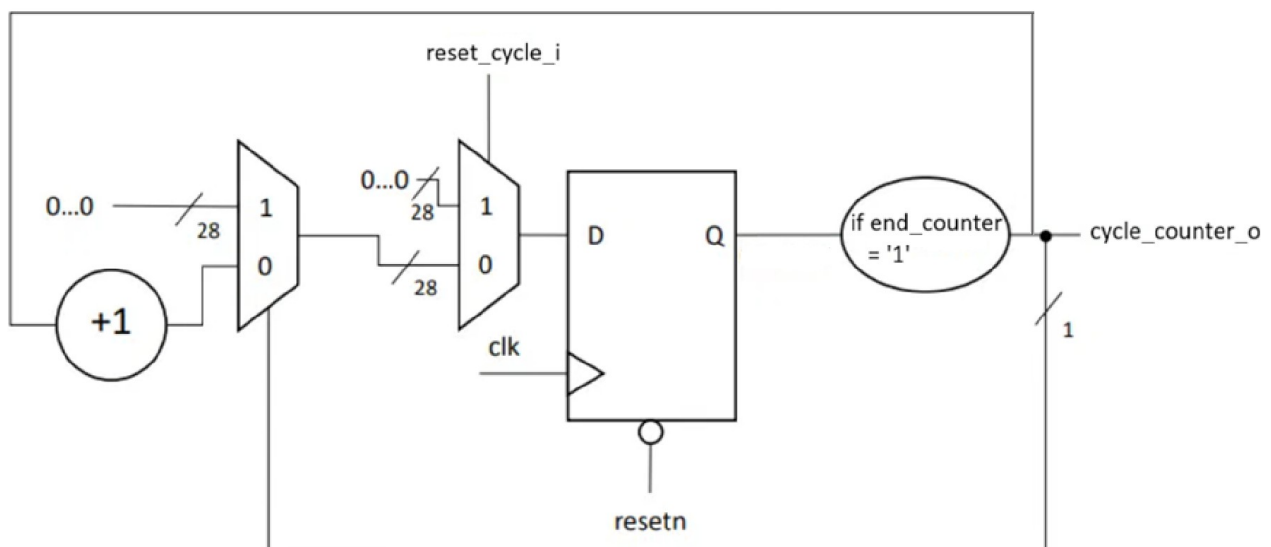


TP3 – Machine à états (FSM)

1. Dans un fichier .vhd, créez un module Counter_unit à partir du compteur du TP1. Le module prendra en entrée un signal d'horloge et de resetn, et donnera en sortie le signal end_counter. Utilisez un paramètre generic() pour définir le nombre de coup d'horloge à compter.

Fait

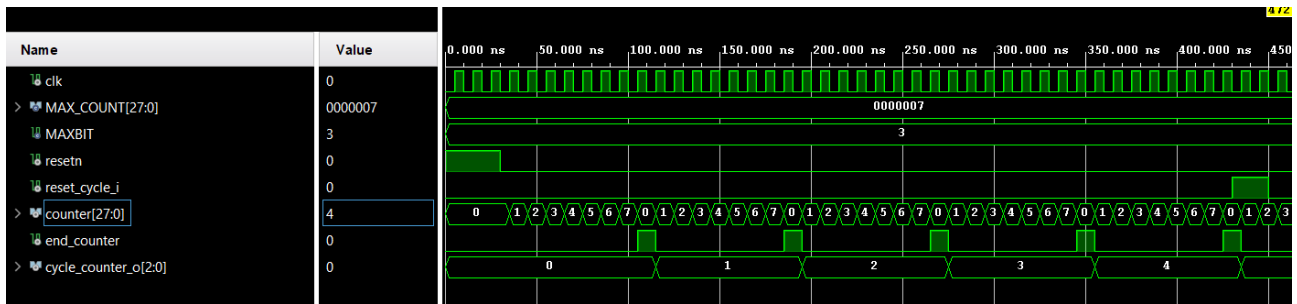
2. En schéma RTL, créez un compteur du signal end_counter. Ce compteur doit permettre de déterminer le nombre de cycles allumé/éteint qui ont été effectués par la LED. Le compteur doit pouvoir être remis à 0, maintenir sa valeur actuelle ou s'incrémenter.



3. Ecrivez un code VHDL décrivant ce compteur de cycle, vous utiliserez le module Counter_unit.

Fait

4. Tester votre architecture avec un testbench



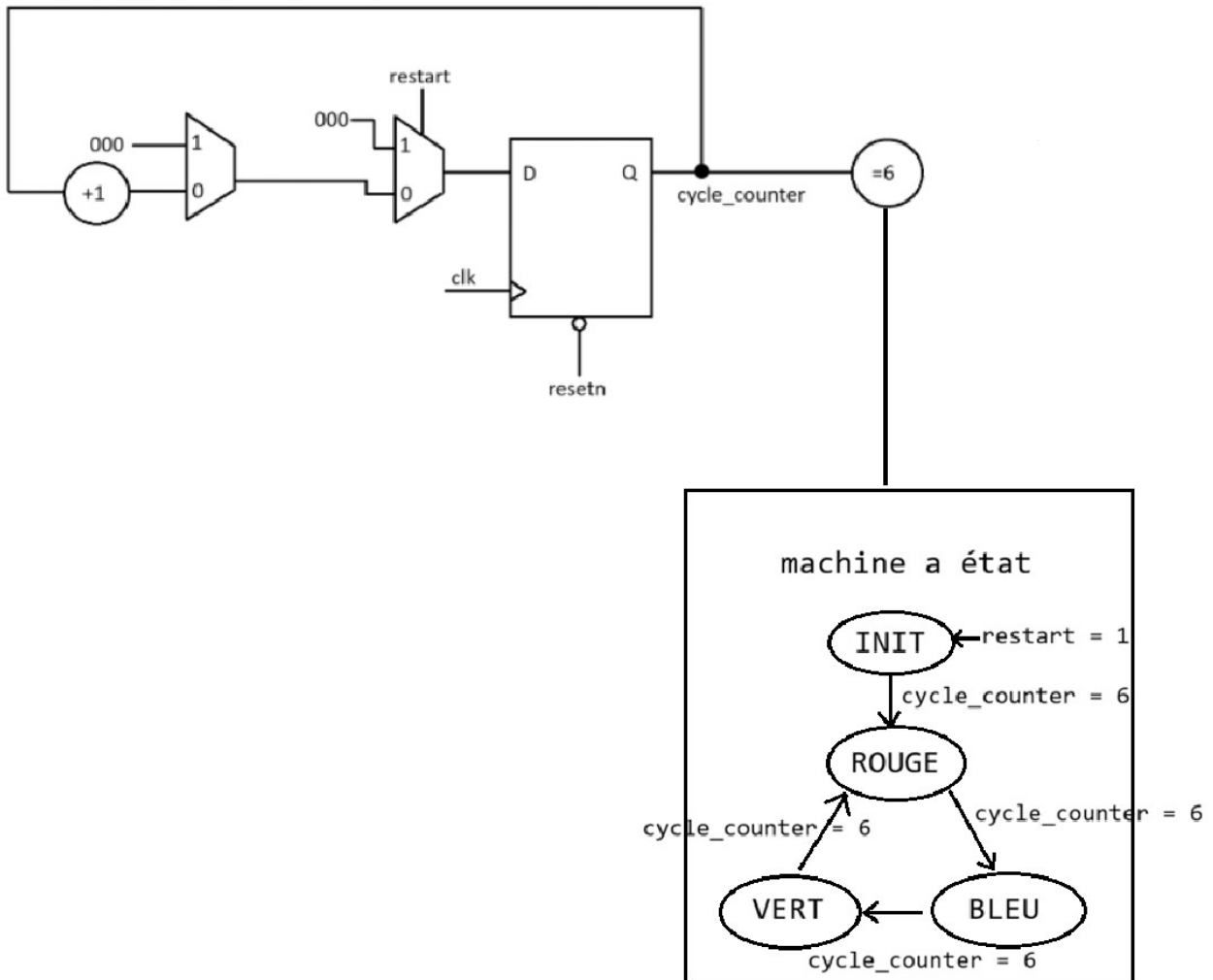
Ici on remarque qu'à chaque coup d'horloge notre compteur de notre unité Counter_unit (counter[27:0]) s'incrémente.

A chaque fois que ce compteur arrive à sa valeur maximale (pour l'instant 7 pour les besoins de la simulation et ensuite 199 999 999) le signal end_counter passe à l'état haut, indiquant une fin de comptage, cycle_counter_o s'incrémente suite à cela, comptant bien le nombre de fois que end_counter passe à l'état haut: cela correspond à un demi-cycle de notre LED (allumée 2 secondes OU éteint 2 secondes).

Afin de faire clignoter une LED 3 fois il suffira de mettre la LED à l'état haut lorsque cycle_counter_o sera paire: cycle_counter_s(0) = 0 (il suffit de regarder le LSB) et la LED à l'état bas lorsque cycle_counter_o sera impaire: cycle_counter_s(0) = 1 TANT QUE cycle_counter_o < 6. En effet, on souhaite faire clignoter 3 fois la LED soit 6 demi_cycle:

cycle_counter_o = 0 (paire) => LED ON
cycle_counter_o = 1 (impaire) => LED OFF
cycle_counter_o = 2 (paire) => LED ON
cycle_counter_o = 3 (impaire) => LED OFF
cycle_counter_o = 4 (paire) => LED ON
cycle_counter_o = 5 (impaire) => LED OFF

5. Créez en RTL une machine à états (FSM) permettant de faire clignoter une LED RGB en rouge puis bleu et enfin en vert avant de recommencer le cycle (rouge, bleu, vert, ...). Dans chaque état la LED devra clignoter 3 fois. De plus, si le bouton restart est appuyé, on retourne dans l'état initial quel que soit l'état dans lequel on se situe. L'état initial est l'état dans lequel on se situe au démarrage, on passe à l'état rouge après 3 clignotements de la LED en blanc (rouge, vert et bleu actifs en même temps).



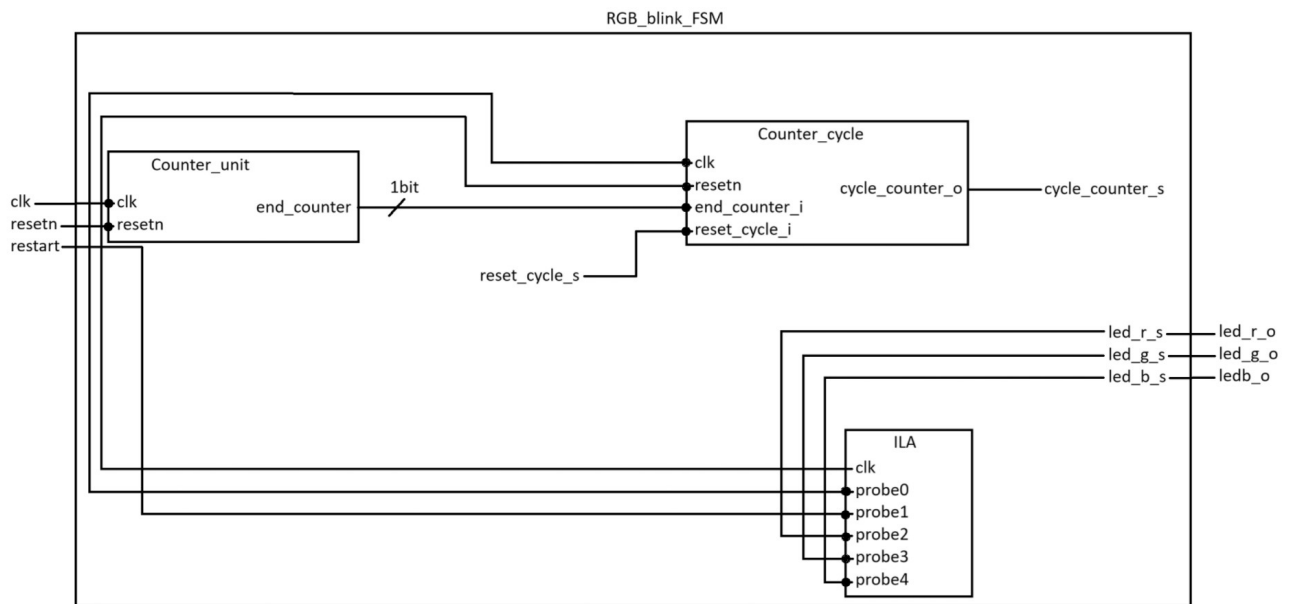
6. Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture.

Entrée:

-clk
-resetn
-restart

Sortie:

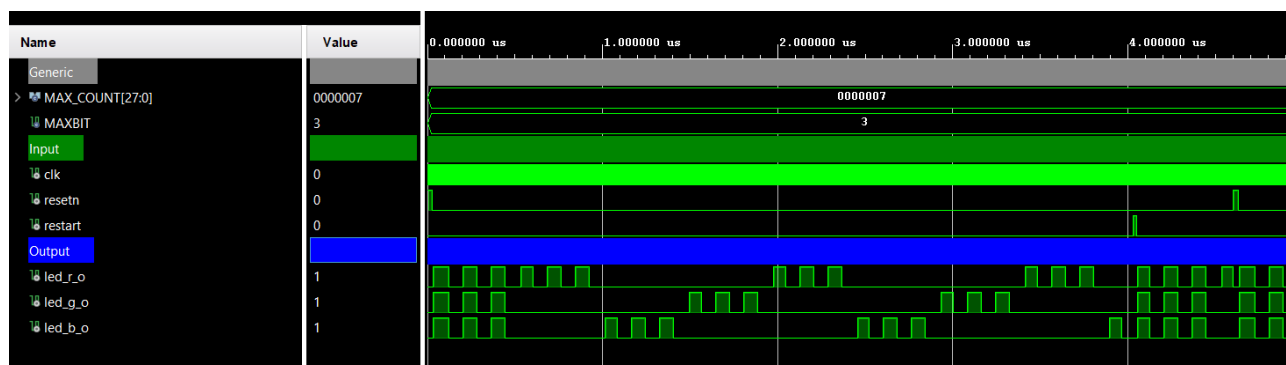
-led_r
-led_g
-led_b



7. Ajoutez à votre code VHDL les éléments que vous venez de créer.

Fait

8. Ecrivez un testbench pour tester votre architecture. Vérifiez à la simulation que vous obtenez le résultat attendu.



On observe bien l'état de sortie des LEDs.

Au début les `led_r_o` (rouge), `led_g_o` (vert) et `led_b_o` (bleu) sont à l'état haut pendant 3 clignottements pour l'initialisation (LED blanche). Vient ensuite le `led_r_o` qui clignotte 3 fois, puis vient le tour de `led_b_o` (bleu) et pour finir `led_g_o` (vert). Ce schéma se répète ainsi de suite jusqu'à ce que le bouton `restart` passe à '1'. Alors on voit qu'il interrompt le clignottement de `led_b_o` et réinitialise le système en allumant la led blanche en faisant clignotter `led_r_o`, `led_g_o` et `led_b_o` en même temps. Vient ensuite le reset qui réinitialise les signaux.

9. Exécutez la synthèse et relevez les ressources utilisées (y compris la FSM). Sur la schématique, identifiez où se situe votre compteur de cycle.

J'ouvre mon rapport de synthèse, je remarque plusieurs éléments:

```
INFO: [Synth 8-802] inferred FSM for state register 'state_reg' in module 'RGB_blink_FSM'
```

State	New Encoding	Previous Encoding
init	00	00
rouge	01	01
bleu	10	10
vert	11	11

Vivado a bien reconnu mon signal `state` (type énuméré `state_t`) en tant que machine à états, et l'a traité avec des optimisations dédiées aux FSM plutôt que comme un simple registre générique.

Je remarque aussi que Vivado a encodé mes états sur 2 bits.

C'est curieux car lors d'une précédente synthèse avec une description VHDL différente, Vivado avait modifié mon encodage sur 4 bits comme suit : (0001, 0010, 0100, 1000).

Il s'agissait d'un encodage "one-hot", apparemment un choix d'optimisation automatique de Vivado

qui permet une logique combinatoire de décodage plus simple et plus rapide, optimisation qu'il n'a visiblement pas souhaité faire sur ce nouveau design.

Voici le détail des composants RTL:

```
Detailed RTL Component Info :
+---Adders :
      2 Input    28 Bit    Adders := 1
      2 Input     3 Bit    Adders := 1
+---Registers :
           28 Bit    Registers := 1
           3 Bit     Registers := 1
           1 Bit     Registers := 4
+---Muxes :
      2 Input     3 Bit    Muxes := 1
      5 Input     2 Bit    Muxes := 1
      2 Input     1 Bit    Muxes := 2
      3 Input     1 Bit    Muxes := 1
      4 Input     1 Bit    Muxes := 3
```

Adders:

Vivado liste tous les additionneurs inférés à partir de mon code RTL, avec leur largeur. Ça correspond bien à mes différents compteurs (Counter_unit interne sur 28 bits, cycle_counter_s sur 3 bits).

Registres:

-28 Bits Registers := 1 => Counter_unit qui compte jusqu'à 199 999 999 qui a besoin de 28 bits.

-3 Bits Registers := 1 => cycle_counter_s

-1 Bit Registers := 4 => end_counter_s + led_r_s + led_g_s + led_b_s.

Multiplexers:

-2 Input 3 Bit Muxes := 1 → le mux hold/increment de Counter_cycle (choix entre cycle_counter_s inchangé et cycle_counter_s + 1, piloté par end_counter_i).

-5 Input 2 Bit Muxes := 1 → la logique de *next-state* du registre state_reg : 4 branches du case (une par état) + la condition prioritaire restart, qui pilotent ensemble les 2 bits du prochain état.

-2 Input 1 Bit Muxes := 2, 3 Input 1 Bit Muxes := 1, 4 Input 1 Bit Muxes := 3 → logique combinatoire à priorités multiples pilotant led_r_s, led_g_s, led_b_s (chaque sortie dépend de plusieurs conditions imbriquées : état courant, cycle_counter_s(0), restart), avec un nombre d'entrées différent selon la complexité de chaque branche if/else par couleur.

Cette liste ne montre pas le registre d'état `state_reg` (2 bits, INIT/ROUGE/BLEU/VERT) : Vivado le classe à part comme FSM (INFO: [Synth 8-802] inferred FSM for state register 'state_reg'), une section distincte qui n'apparaît pas dans cette capture. On sait qu'il existe bel et bien car Report Cell Usage du même rapport indique $FDCE := 37$, et $28 + 3 + 4 + 2$ (`state_reg`) = 37 correspond exactement:

Report Cell Usage:

	Cell	Count
1	ila	1
2	BUFG	1
3	CARRY4	7
4	LUT1	1
5	LUT3	2
6	LUT4	33
7	LUT5	5
8	LUT6	4
9	FDCE	37
10	IBUF	3
11	OBUF	3

On remarque mon ILA pour le debug.

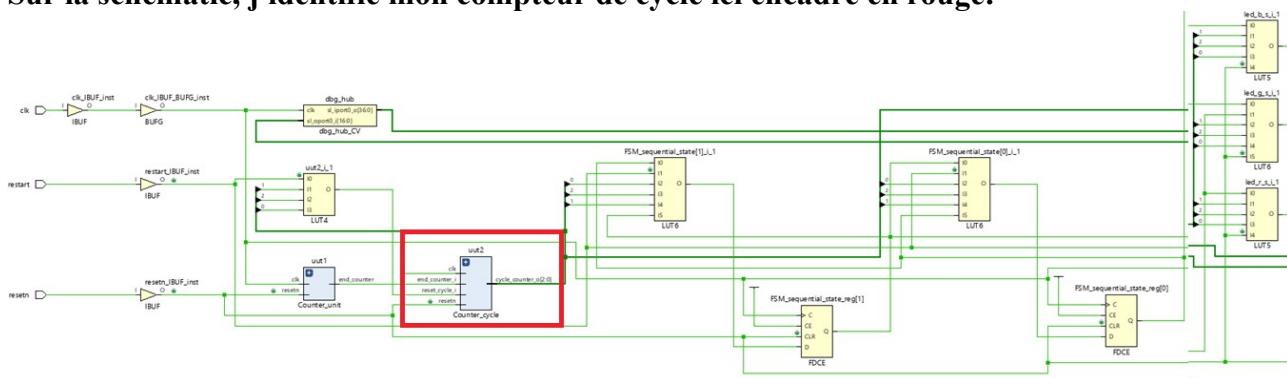
BUFG pour transmettre le signal de l'horloge

Les LUT permettant de réaliser toutes les opérations logiques (compteur, comparateur, machine états)

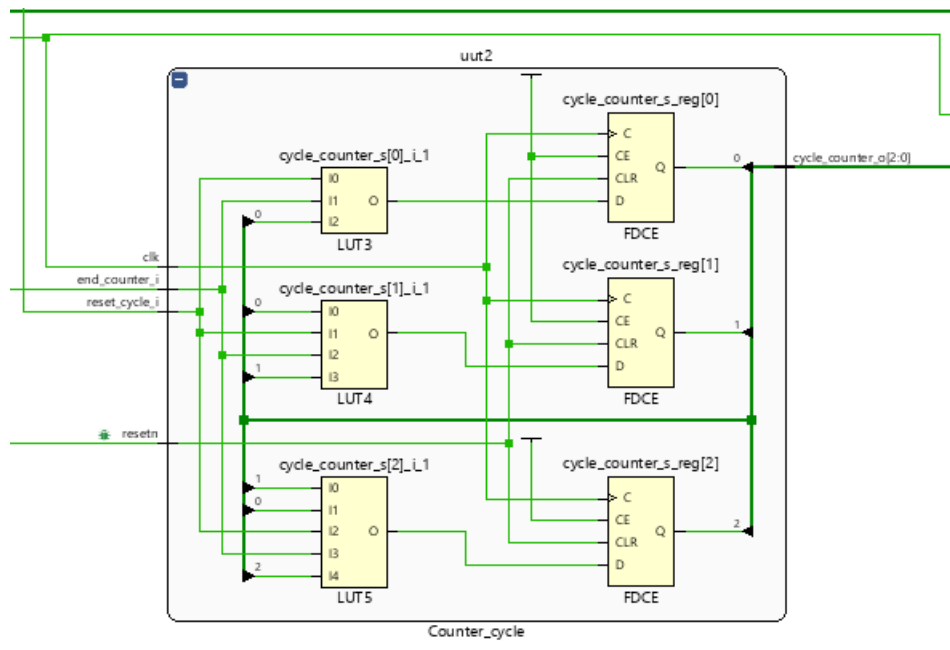
Le rapport confirme que la synthèse s'est bien déroulée, l'utilisation de ressources reste très modeste: quelques dizaines de LUT/bascules

Cela est cohérent avec la simplicité du design. Rien d'anormal ne saute aux yeux ici.

Sur la schématique, j'identifie mon compteur de cycle ici encadré en rouge:



Il s'agit du composant que j'ai instancié dans mon top, je peux l'ouvrir pour voir comment il est implémenté:



On remarque que le synthétiseur a préféré faire n étage par bit, LUT + FDCE

- cycle_counter_s[0]_i_1 (LUT3) => cycle_counter_s_reg[0] (FDCE)
- cycle_counter_s[1]_i_1 (LUT4) => cycle_counter_s_reg[1] (FDCE)
- cycle_counter_s[2]_i_1 (LUT5) => cycle_counter_s_reg[2] (FDCE)

Chaque bit a sa propre bascule, et leurs sorties Q (0, 1, 2) forment ensemble le bus cycle_counter_o[2:0].

10. Modifiez le fichier de contraintes pour connecter vos entrées / sorties du système avec les broches de la carte. Réglez l'horloge pour que sa fréquence soit à 100MHz.

Je décommente les lignes correspondantes aux contraintes de l'horloge:

```
6 | # PL System Clock
7 | set_property -dict { PACKAGE_PIN H16   IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L13P_T2_MRCC_35 Sch=sysclk
8 | create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk }]; #set
9 |
```

J'affecte les LED_s dont nous avons besoin aux pins de la carte:

```
# RGB LEDs
set_property -dict { PACKAGE_PIN L15   IOSTANDARD LVCMOS33 } [get_ports { led_b }]; #IO_L22N_T3_AD7N_35 Sch=led0_b
set_property -dict { PACKAGE_PIN G17   IOSTANDARD LVCMOS33 } [get_ports { led_g }]; #IO_L16P_T2_35 Sch=led0_g
set_property -dict { PACKAGE_PIN N15   IOSTANDARD LVCMOS33 } [get_ports { led_r }]; #IO_L21P_T3_DQS_AD14P_35 Sch=led0_r
```

Pour finir, j'affecte le reset et le restart à deux boutons sur la carte:

```
# Buttons
set_property -dict { PACKAGE_PIN D20   IOSTANDARD LVCMOS33 } [get_ports { resetn }]; #IO_L4N_T0_35 Sch=btn[0]
set_property -dict { PACKAGE_PIN D19   IOSTANDARD LVCMOS33 } [get_ports { restart }]; #IO_L4P_T0_35 Sch=btn[1]
```

11. Lancez l'implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et de hold et identifiez le chemin critique).

Implementation Complete 

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1,746 ns	Worst Hold Slack (WHS): 0,020 ns	Worst Pulse Width Slack (WPWS): 2,750 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 3843	Total Number of Endpoints: 3827	Total Number of Endpoints: 2152
All user specified timing constraints are met.		

Notre WNS (chemin critique) a une marge de 1,746 ns ce qui est assez confortable, cohérent avec la simplicité de notre design.

Timing x													
Intra-Clock Paths - sys_clk_pin - Setup													
Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 21	1.746	5	14	<hidden>	<hidden>	5.855	1.076	4.779	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 22	1.746	5	14	<hidden>	<hidden>	5.855	1.076	4.779	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 23	1.747	5	14	<hidden>	<hidden>	5.853	1.076	4.777	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 24	1.747	5	14	<hidden>	<hidden>	5.853	1.076	4.777	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 25	1.837	5	14	<hidden>	<hidden>	5.762	1.076	4.686	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 26	1.837	5	14	<hidden>	<hidden>	5.762	1.076	4.686	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 27	1.837	5	14	<hidden>	<hidden>	5.762	1.076	4.686	8.0	sys_clk_pin	sys_clk_pin		0.035

On remarque que sur 5,855 ns de délai total, 4,779 ns (soit 82%) viennent du routage (Net Delay) contre seulement 1,076 ns de logique pure (Logic Delay). Ça signifie que le "coût" de ce chemin vient principalement de la distance physique parcourue sur le FPGA, pas de la logique.

12. Générez le bitstream pour vérifier le système sur carte.

Démonstration:

<https://youtube.com/shorts/bnu1BYRqaEE?feature=share>